第 3 讲：哈希思想、哈希表、`set` 与 `dict`

0. 本讲的核心问题

给定一串数据，我们经常要回答：

如果每次都在线性列表中查找，单次查询最坏是 `O(n)`。如果一边遍历，一边做 `n` 次这样的查询，总时间可能变成 `O(n²)`。

哈希表的目标是：

> 通过键本身快速推断它应该存放在哪里，使插入、查询和删除在平均情况下接近 `O(1)`。

Python 的 `set` 和 `dict` 都主要建立在哈希表上：

```
├── set
└── dict      →
```

---

1. 从直接寻址开始

假设键只可能是 `0` 到 `999` 的整数。可以准备一个长度为 1000 的数组：

```
[python]
present = [False] * 1000
present[42] = True
```

查询 `42` 是否出现：

```
[python]
present[42]
```

时间是 `O(1)`，因为键本身就是下标。

但如果键可能是：

```
10^18
"alice@example.com"
(-3, 8)
```

就不能为所有可能的键准备槽位。我们需要把一个很大的键空间压缩到有限数量的槽位中。

---

2. 哈希函数与槽位

哈希函数把键转换成整数：

$$ h(key) \rightarrow \text{integer} $$

再根据表的容量映射到一个槽位：

$$ index=h(key)\bmod m $$

其中 `m` 是槽位数量。

直觉示意：

```
"cat"  ──hash──> 18493 ──mod m──>    3
"dog"  ──hash──> 72011 ──mod m──>    7
```

查询时再次执行相同计算，因此不必从头扫描全部元素。

Python 会负责具体的哈希、槽位布局、扩容和冲突处理。做题时我们主要使用它提供的接口。

---

3. 哈希冲突为什么必然存在

可能的键远多于槽位，根据抽屉原理，不同键必然可能映射到同一槽位：

```
key_a ──>    5
key_b ──>    5
```

这叫哈希冲突。

发生冲突不代表两个键相等。哈希表仍需要比较实际的键，并用额外规则寻找或保存它们。常见策略包括链地址法和开放寻址法；Python 的具体实现细节会随版本演进，本课程不要求复现。

你需要掌握的结论：

- 哈希值相同，键不一定相等；
- 相等的键必须拥有相同的哈希值；
- 冲突越多，查询通常越慢；
- 表过满时需要扩容；
- 因此查询、插入、删除通常说“平均 `O(1)`”。

理论最坏情况可能退化，但常规 LeetCode 分析采用平均复杂度。

---

4. 为什么键必须可哈希

键放入哈希表后，系统依赖它的哈希值再次找到原位置。如果键在存储期间发生改变，哈希值也可能改变，原来的位置就失去意义。

因此，Python 哈希表的键通常必须是不可变、可哈希的对象。

通常可以作为键：

```[python]
42
"hello"
(3, 5)
```

通常不能作为键：

```[python]
[3, 5]       # list
{3, 5}       # set
{"x": 3}    # dict
```

元组只有在内部元素也都可哈希时才能作为键：

```[python]
(1, 2)          #
(1, [2, 3])     #
```

二维坐标经常用元组作键：

```[python]
```

```
visited = set()
visited.add((row, col))
```

---

5. `set`：只保存键

5.1 常用操作

```
[python]
seen = set()

seen.add(5)
seen.add(8)
seen.add(5)        #         5

5 in seen          # True
9 not in seen      # True

seen.remove(5)     #       KeyError
seen.discard(5)    #
```

平均复杂度：

操作	平均时间
`x in seen`	`O(1)`
`seen.add(x)`	`O(1)`
`seen.remove(x)`	`O(1)`

集合自身可能保存 `n` 个元素，所以额外空间通常是 `O(n)`。

5.2 集合运算

```
[python]
a = {1, 2, 3}
b = {2, 3, 4}

a & b   #    {2, 3}
a | b   #    {1, 2, 3, 4}
a - b   #    {1}
```

做题时不要只因为存在集合运算符就使用它。仍要分析操作需要遍历多少元素、是否创建新集合。

---

6. `dict`：保存键到值的映射

6.1 常用操作

```
[python]
positions = {}
positions[10] = 3
positions[20] = 7

10 in positions       #     True
positions[10]         #     3
positions.get(99)     #        None
positions.get(99, -1) #        -1
```

注意：

```
[python]
x in positions
```

检查的是键，不是值。

6.2 遍历字典

```python
for key in positions:
    print(key)

for value in positions.values():
    print(value)

for key, value in positions.items():
    print(key, value)
```

现代 Python 的 `dict` 保留插入顺序，但它不是自动按键排序的。算法正确性不应误依赖“看起来刚好有序”。

---

7. 高频模式一：是否出现过

问题：判断数组是否存在重复元素。

```python
def contains_duplicate(nums):
    seen = set()

    for x in nums:
        if x in seen:
            return True
        seen.add(x)

    return False
```

循环不变量：

> 处理当前 `x` 之前，`seen` 恰好包含此前已经处理的元素。

必须先检查、后加入。如果先加入，当前元素一定已经存在。

- 平均时间：`O(n)`；
- 额外空间：`O(n)`。

---

8. 高频模式二：频率统计

问题：统计每个字符出现次数。

```python
def character_counts(text):
    counts = {}

    for ch in text:
        counts[ch] = counts.get(ch, 0) + 1

    return counts
```

这一句可以拆开理解：

```python
old_count = counts.get(ch, 0)
new_count = old_count + 1
counts[ch] = new_count
```

循环不变量：

> 处理位置 `i` 之前，`counts` 正确统计了 `text[:i]` 中每个字符的出现次数。

Python 还提供：

```
[python]
from collections import Counter
counts = Counter(text)
```

学习阶段建议先能手写普通 `dict` 版本，再使用 `Counter`。

---

9. 高频模式三：键映射到附加信息

两数之和

给定 `nums` 和 `target`，寻找：

$$ nums[i]+nums[j]=target,\quad i<j $$

处理当前值 `x=nums[j]` 时，需要的值唯一确定：

$$ needed=target-x $$

如果只使用 `set`，能知道 `needed` 是否出现，却不知道它的下标。于是使用 `dict`：

```
[python]
def two_sum(nums, target):
    positions = {}

    for j, x in enumerate(nums):
        needed = target - x

        if needed in positions:
            return [positions[needed], j]

        positions[x] = j
```

循环不变量：

> 处理下标 `j` 前，`positions` 保存了此前元素的“值 → 某个下标”。

为什么必须先查询、后写入？考虑：

```
[python]
nums = [3, 3]
target = 6
```

在处理第一个 `3` 时不能用它与自己配对。先查询此前元素，天然保证返回的下标小于当前下标。

---

10. 高频模式四：把对象变成规范键

问题：判断两个字符串是否为字母异位词。

```
"anagram"   "nagaram"
```

一种思路是统计频率；另一种思路是把每个字符串转化为规范形式：

```
[python]
key = "".join(sorted(word))
```

所有由相同字母组成的字符串会得到相同的键。

分组异位词：

```
[python]
def group_anagrams(words):
    groups = {}

    for word in words:
        key = "".join(sorted(word))

        if key not in groups:
            groups[key] = []

        groups[key].append(word)

    return list(groups.values())
```

这里的设计问题不是“是否使用字典”，而是：

> 什么键能够准确代表一个分组？

如果单词平均长度为 `k`，排序每个单词需要 `O(k log k)`，总时间约为 `O(nk log k)`。

---

11. 进阶模式：最长连续序列

给定无序数组：

```
[100, 4, 200, 1, 3, 2]
```

最长连续序列是：

```
1, 2, 3, 4
```

长度为 4。

如果从每个数字都向后搜索，会重复工作。例如从 `1 2 3 4` 分别出发。

关键观察：只有当 `x-1` 不存在时，`x` 才可能是一个序列的起点。

```
[python]
def longest_consecutive(nums):
    values = set(nums)
    best = 0

    for x in values:
        if x - 1 not in values:
            current = x
            length = 1

            while current + 1 in values:
                current += 1
                length += 1

            best = max(best, length)

    return best
```

虽然有嵌套的 `while`，总平均时间仍是 `O(n)`：只有序列起点会启动向后搜索，每个连续元素只在其所属序列中被访问一次。

这再次说明：不能只数循环层数，要计算最内层操作总共执行多少次。

---

12. 常见错误

12.1 把值和下标混淆

```
[python]
for i, x in enumerate(nums):
    positions[i] = x       #    →
```

两数之和通常需要：

```
[python]
positions[x] = i           #   →
```

写字典前先用一句话说出“键表示什么，值表示什么”。

12.2 更新顺序错误

同一个结构中，“先查后写”和“先写后查”可能产生不同结果。必须根据不变量决定顺序。

12.3 使用不可哈希的键

需要把列表状态作为键时，通常转换为元组：

```
[python]
key = tuple(values)
```

12.4 忘记空间代价

把列表转换为集合：

```
[python]
values = set(nums)
```

平均查询更快，但额外空间是 `O(n)`。

12.5 误以为字典自动排序

保留插入顺序不等于按键大小排序。

---

13. 选择 `set` 还是 `dict`

先问：

> 除了“这个键是否存在”，我还需要保存什么信息吗？

需求	结构
是否出现过	`set`
去重	`set`
值对应下标	`dict`
元素出现次数	`dict`
分类键对应一组元素	`dict`
坐标是否访问过	`set[tuple]`

---

14. 本讲编码检查表

```
1.
2.      dict
3.
4.
5.
6.
7.
8.
9.
10.
```

15. 本讲总结

1. 哈希表通过键的哈希值快速定位存储位置。
2. 冲突不可避免，所以查询复杂度通常表述为平均 `O(1)`。
3. `set` 是只保存键的哈希集合；`dict` 保存键和值。
4. 键必须可哈希，列表、集合和字典通常不能直接作为键。
5. 高频模式包括：存在性、频率、值到下标、规范键分组。
6. 解题的核心仍然是定义状态和不变量，而不是机械地“看到数组就套哈希表”。